Jury prep: showing the code and answering questions

Open this on your phone during the event. Part 1 is what to open, Part 2 the 7-stop code tour, Part 3 the questions they are most likely to ask and what to say, Part 4 what to be honest about.

---

1. Before they arrive (10 minutes)

1. Laptop on the charger, Windows power mode **Best performance**, notifications off.
2. In VS Code, open these files as tabs **in this order** (left to right), so you just click through:
    1. `docs/READ_FIRST.md` (the system in plain words)
    2. `backend/worker/capture.py`
    3. `backend/worker/run_worker.py`
    4. `backend/worker/rules.py`
    5. `backend/app/services/escalation.py`
    6. `backend/app/services/call_agent.py`
    7. `backend/app/services/faces.py`
    8. `backend/app/services/assistant.py`
    9. `docs/CONTRACT.md` and `docs/disclosures.md`
3. One terminal in `backend\` with the venv on, ready to type `pytest -q`.
4. The website open on **Live Feed**, one camera running, **Demo alert** visible.
5. `CALLS_MODE=live` only if your phone is on you and the numbers on the call list belong to your team.

Font: press `Ctrl` `+` twice in VS Code so the jury can read it.

---

2. The code tour (5 minutes, 7 stops)

Say the one-line summary first, then show the code. Jump to a line with `Ctrl+G` and the line number.

#	Open	Line	Say this
1	`backend/worker/capture.py`	`class LatestFrameCapture` (line 45)	"Each camera has its own process. This thread reads the camera nonstop and keeps only the newest frame, so the AI never analyses old video, even if it is slow."

#	Open	Line	Say this
2	`backend/worker/run_worker.py`	`def run` (215), the loop at 270	"This is one camera's loop: people and pose (YOLO11), fire/smoke, helmet, faces, then the rules. The live view is a separate thread at 25 fps (`class LiveView` , 166), so video stays smooth while the AI works."
3	`backend/worker/rules.py`	`class RulesEngine` (210), `_streak` (395), `_apply_cooldown` (429)	"Detections are not alarms. The rules decide: a person's feet in a restricted zone for N seconds, fire in 3 of the last 6 frames, a fall only if the torso is flat **and** the hips dropped. One wrong frame cannot raise an alarm. Every threshold comes from settings, not the code."
4	`backend/app/services/escalation.py`	`ladder_for` (64), `_nobody_answered` (256)	"Who gets called depends on severity. Critical: Head Guard, Admin, Manager at once. If nobody acknowledges in 45 seconds, every guard on shift gets a call and a WhatsApp. It stops at the first acknowledgement from any channel."
5	`backend/app/services/call_agent.py`	`opening` (148), `decide` (248), `ACTIONS` (41)	"The phone call is a conversation. The LLM only returns `{reply, action}` and the action must be in this allowed list, so the AI can never do something we did not allow. A critical incident can only be acknowledged by phone, never closed."
6	`backend/app/services/faces.py`	`class FaceEngine` (55), `class FaceBank` (118)	"Face recognition for restricted areas runs on the laptop. We keep only a 128-number fingerprint per photo, never the photo, and only with the person's consent. Allowed people walk in, anyone else raises an alarm."
7	`backend/app/services/assistant.py`	`class IncidentFilter` (43), `ask` (177)	"Friday answers questions about incidents. The LLM only fills this filter; our code builds the database query. The LLM never writes SQL and never invents numbers."

Finish with the proof: run `pytest -q` in the terminal. It runs about 300 tests in 2 minutes and they pass.

---

3. The questions they will most likely ask

"Show me where the AI model is."

- The models: `backend/models/` (`yolo11n-pose.pt`, `fire_smoke_v2.pt`, `ppe.pt`, `rider_helmet.pt`, and the `*_openvino_model` folders, the fast Intel versions).
- Where they load and run: `backend/worker/detectors.py` (`PersonPoseTracker`, `ClassDetector`, `optional_detector`), called from `backend/worker/run_worker.py`.
- How they were trained: `backend/tools/vision_training/README.md` (Colab steps).

"Did you train your own model?"

Yes, three, on Google Colab (free T4 GPU), starting from YOLO11n: - **Fire/smoke:** D-Fire dataset (about 21,000 images) plus about 200 photos of our own lighter that we filmed and labelled. mAP50 0.66 overall (smoke 0.73, fire 0.59). - **PPE:** Construction-PPE, helmet and vest mAP50 0.83. - **Rider helmet:** a 984-image Roboflow dataset.

Person detection and pose use the standard pretrained YOLO11n-pose. Face recognition uses OpenCV's pretrained YuNet and SFace models. We did not train those.

"What does mAP50 mean?"

"Of the boxes the model draws, how well they match the real objects, averaged over classes, counting a box as right when it overlaps the true box by at least half. 1.0 is perfect."

"How do you avoid false alarms?"

1. **Rules on top of detections:** an alarm needs several sightings in a row (fire in 3 of the last 6 analysed frames, a person in a zone for N seconds, a fall that stays down 5 s).
2. **A cooldown:** the same camera cannot raise the same alarm again for a set time.
3. **Severity:** low things (no PPE, loitering) only send a WhatsApp, never a call.
4. **A human decides:** the call asks the guard; critical incidents can only be closed on the dashboard.

"What happens in the first seconds after a fire?"

"The camera worker confirms it in about a second, saves a snapshot and a 10-second clip, and tells the API. The dashboard shows it in under 2 seconds, Friday announces it, and the phones of the Head Guard, Admin and Manager ring at once. If nobody acknowledges in 45 seconds, every guard on shift is called."

"Where does the video go? Is it sent to the cloud?"

"No. Video and face fingerprints never leave the laptop. Only short text goes out: the alert text to Twilio for the call and WhatsApp, and the question text to Groq for the LLM. Offline, we fall back to a local model (Ollama) and local speech-to-text."

"What about privacy and faces?"

"Face enrollment needs the person's consent (the app refuses without it). We store only a fingerprint of 128 numbers, not the photo. A person can be deleted with one button. Face

recognition only runs inside restricted areas, not everywhere. Every action is in the audit log."

"How does it scale to 50 cameras?"

"Each camera is its own process, so cameras do not slow each other down and one crash does not stop the others. One laptop handles about 2 to 3 cameras at 6 to 8 analysed frames per second. For more, add a small PC or a GPU box per group of cameras; they all report to the same API. Twilio places about 1 call per second, so a roll call of 20 guards takes about 20 seconds."

"What if the internet or Twilio is down?"

"The dashboard and the alarm sound still work on the local network. Every Twilio, WhatsApp and Groq call has a timeout and one retry; if Twilio fails, the dashboard and WhatsApp still alert, and the Calls light at the top turns red with Twilio's reason. The LLM falls back to the offline model."

"Why phone calls? Nobody uses phone calls."

"Because a dashboard needs someone watching it, and at 2 a.m. nobody is. A ringing phone wakes a guard. That is our pitch: every factory has cameras nobody watches, most AI adds a dashboard nobody watches, ours picks up the phone."

"How does the AI on the call know what to do?"

Show `call_agent.py`. "She gets the incident facts and the conversation. The LLM returns a reply and one action from a fixed list. Our code checks the action is allowed for that call and does it. 'I'm going' is understood instantly without the LLM."

"Is this secure?"

- PIN login, sessions of 12 hours, 5 tries per minute.
- Each user sees only their cameras (`backend/app/services/permissions.py`, `backend/app/deps.py`).
- Camera streams and media links are signed and expire (HMAC, `permissions.py`).
- Twilio webhooks are checked with Twilio's signature (`verified_form` in `backend/app/routers/twilio.py`).
- Through ngrok only the Twilio and share paths are reachable; the dashboard stays local (`backend/app/main.py`).
- Secrets live only in `.env`, never in Git (`.env.example` has empty values).

"Show me the database."

- Tables: `backend/app/models.py` (incidents, timeline, calls, users, zones, faces, conversations, audit log).
- Migrations: `backend/alembic/versions/`.
- PostgreSQL, run locally.

"Show me the API."

`docs/CONTRACT.md` lists every endpoint. Also open `http://localhost:8000/docs` in the browser: FastAPI's live, clickable API page.

"How did you test it?"

`backend/tests/` has about 300 automatic tests: the rules engine with fake detections, the call ladder, the phone conversation, face matching, permissions, the API. Run `pytest -q` live. They use a separate test database.

"Did you write all this yourselves?" (be honest)

"We designed the system and made every decision, and we used an AI coding assistant, Claude Code, to write a lot of the code, which we reviewed, ran and tested. It is listed in `docs/disclosures.md`, as the rules ask. I can walk you through any part of it." Then pick any stop from Part 2 and explain it in your own words. Practise this the night before: the jury cares that **you understand it**.

"What was the hardest part?"

Pick one you lived through: - Making a small lighter flame count as fire (we filmed and labelled our own photos and retrained). - Making the phone call feel like a conversation and fast (we answer "I'm going" instantly, and the opening sentence is made while the phone rings). - Old camera processes that kept running after restarts (now they stop with the API).

"What would you do next?"

- A "Suspected fire" pre-alarm: instant amber warning on the first sighting, calls only once confirmed.
- A cheap IoT smoke sensor (ESP32) to confirm what the camera sees.
- More training data from real factory cameras, and a GPU box for many cameras.

---

4. Be honest about these (it earns trust)

- **The fire model is good, not perfect.** On the D-Fire validation set it scores 0.66 mAP50. It can confuse glowing smoke with fire; that is why the rules need several frames and a human confirms on the call.
- **On a laptop without a GPU** we analyse about 6 to 8 frames per second per camera, enough for 2 to 3 cameras.
- **A phone takes 3 to 6 seconds to start ringing.** That is the phone network, not our software.
- **Some tests are only in simulation** (the wake word with a real microphone, very large camera counts).
- **Never claim** it detects everything or replaces guards. It makes sure a human is told, fast.

---

5. If something breaks during the demo

Problem	What to do
A camera shows offline	Cameras page, Restart. Phone camera: check the IP Webcam address (Edit).

Problem	What to do
No phone rings	Look at the Calls light: amber = pretend mode, red = hover for Twilio's reason. Use **Demo alert** and show the timeline and Conversations page instead.
Fire not detected live	Use **Demo alert, Fire**: it shows the same alarm, calls and timeline.
Friday does not hear	Type the question on Friday's page, or hold Space.